

Rapport de Projet : Chatbot

Système RAG avec Recherche Hybride (BM25 + FAISS)

Réalisé par : EL MOUSS Jamaa

Sous la direction du **Prof. Hassan Al Moatassime**

Département Mathématiques

13 / janvier / 2026



- 1 Introduction
- 2 Gestion et Préparation des Données
- 3 Architecture et Fonctionnement du Système RAG
- 4 Conclusion



Problème traité :

- **Difficulté pour les étudiants** d'accéder rapidement à des informations précises sur une filière ou un parcours académique, telles que :
 - Objectifs du programme
 - Contacts des responsables de la filière
 - Conditions d'admission et prérequis
 - Modules étudiés
- Les informations sont dispersées dans de nombreux documents PDF, rendant leur recherche longue et fastidieuse.



Motivation / Intérêt du sujet :

Objectif : Créer un Chatbot(RAG) intelligent capable de centraliser et d'organiser l'information.

- ⚡ **Réponses instantanées .**
- ⚙️ Conversion des documents complexes en **données JSON structurées** pour une précision maximale.

-  **Source** : Site officiel de la **FST de Marrakech**
-  **Documents** : Brochures PDF (Licences, Masters, Ingénieurs)
-  **Extraction brute** :
 - Utilisation de **PyPDF2** pour la lecture des fichiers
 - Nettoyage automatique par **Regex** (suppression des bruits, headers, et numéros de page).

Le système transforme le texte non structuré en une base de données JSON organisée :

- **LLaMA 3.1 (Groq)** pour extraire : objectifs, compétences et modules
- Structuration finale en **JSON** enrichi
- **Enrichissement sémantique** : Chaque segment de texte est lié au nom et à l'identifiant de la filière pour éviter toute confusion.

Pour permettre la recherche sémantique, les données subissent un traitement mathématique :

- **Génération d'Embeddings** : Utilisation du modèle all-MiniLM-L6-v2 pour transformer le texte en vecteurs.
- **Indexation FAISS** :
 - Stockage des vecteurs dans un index **IndexFlatIP**.
 - Recherche de similarité cosinus pour trouver les segments les plus proches de la question
- **Persistence** : Sauvegarde de l'index sur disque pour un chargement instantané.



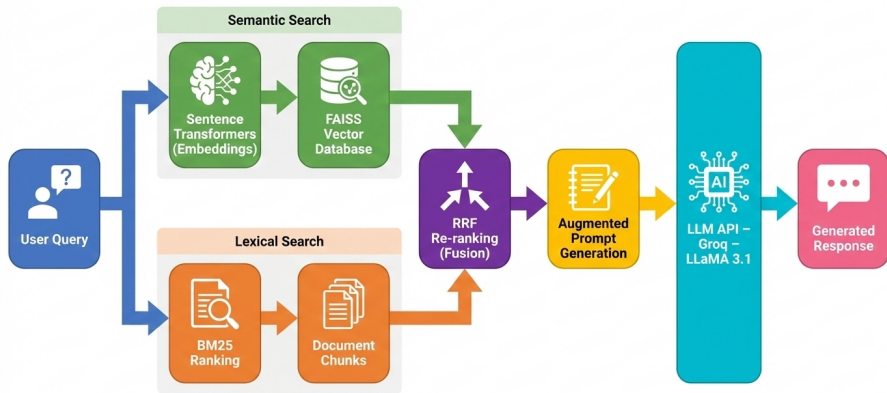
Définition

Le **Retrieval-Augmented Generation (RAG)** est une architecture qui combine la **recherche d'informations** dans une base documentaire avec la **génération de texte** par un modèle de langage.

- ? L'utilisateur pose une question
- 🔍 Le système recherche les passages pertinents
- 🗣️ Le modèle de langage génère une réponse basée uniquement sur ces passages

Hybrid RAG Architecture

Hybrid RAG Architecture

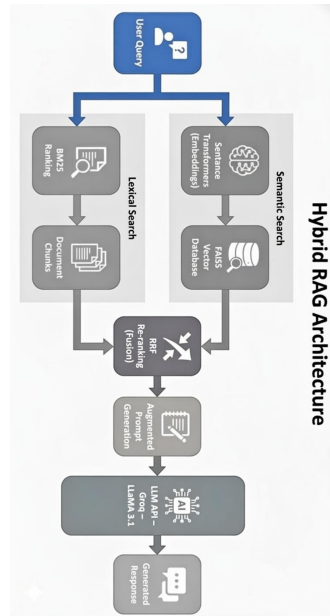




Explication de l'Architecture RAG

L'intention (User Query) :

C'est le moment où l'utilisateur pose sa question en langage naturel (par exemple : "Comment fonctionne le RAG ?").

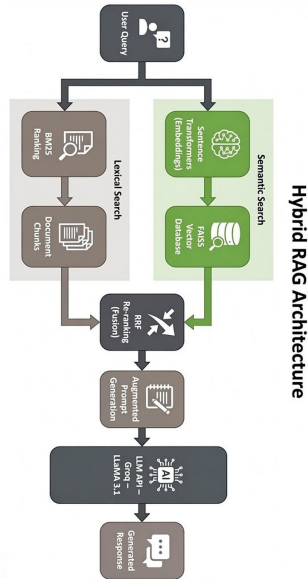




Q Recherche Sémantique (Semantic Search) :

Cette branche se concentre sur le **sens** de la question plutôt que sur les mots exacts.

-  **Embeddings** : La question est convertie en vecteur numérique (mathématique).
-  **FAISS** : Une base de données vectorielle trouve les documents qui ont un sens similaire (voisins).

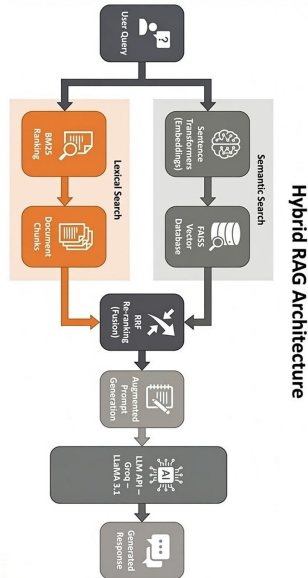


Explication de l'Architecture RAG

Recherche Lexicale (Lexical Search) :

Cette branche utilise une approche traditionnelle pour trouver des correspondances de **mots-clés exacts**.

- **BM25 Ranking** : Un algorithme statistique (évolution de TF-IDF) qui classe les documents selon la fréquence des mots de la question.
- **Utilité** : Crucial pour trouver des termes précis (noms propres, acronymes, codes) que la recherche sémantique pourrait manquer.





Explication de l'Architecture RAG

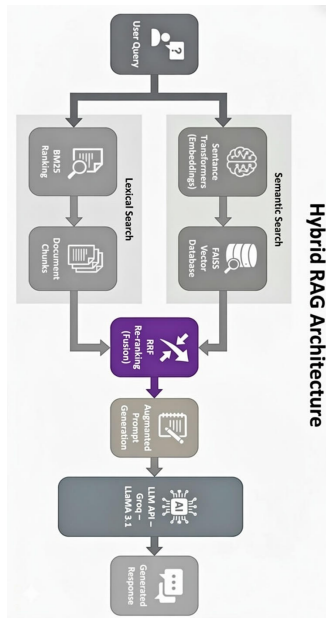
⚙️ Fusion RRF (Reciprocal Rank Fusion) :

Harmonisation des résultats via la formule :

$$\text{Score} = \frac{\beta}{k + \text{rang}_{\text{BM25}}} + \frac{1 - \beta}{k + \text{rang}_{\text{FAISS}}}$$

≡ Paramètres :

- β : Pondération (0 à 1) pour équilibrer l'importance.
- k : Constante de lissage (ex : 60) pour éviter les biais.



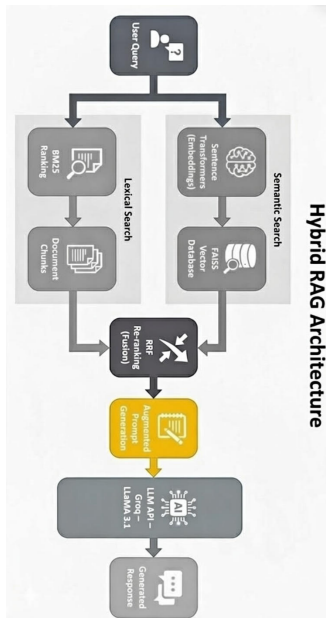


Explication de l'Architecture RAG

Génération de Prompt Augmenté

Construction du "dossier" final pour l'IA :

-  **La Recette :**
Prompt = Question +
Top Documents (Contexte)
-  **Le Garde-fou :**
Ajout d'une consigne stricte :
*"Réponds uniquement avec ces
infos"* pour éviter les
hallucinations.



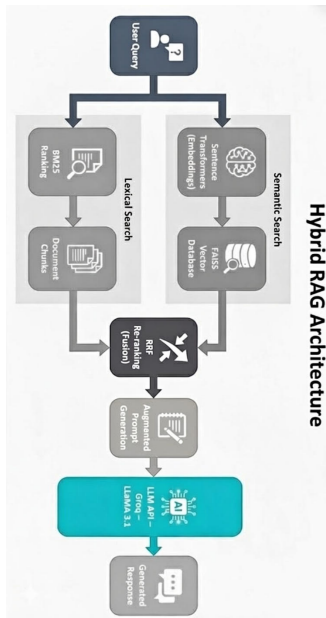


Explication de l'Architecture RAG

API LLM (Groq & LLaMA 3.1)

C'est le moteur d'intelligence qui rédige la réponse finale :

- 🔧 **Le Moteur (Groq) :**
Utilisation de puces spécialisées (LPU) pour une génération de texte ultra-rapide (faible latence).
- 🧠 **L'Intelligence (LLaMA 3.1) :**
Le modèle analyse le contexte validé et formule une réponse naturelle, fluide et synthétique.

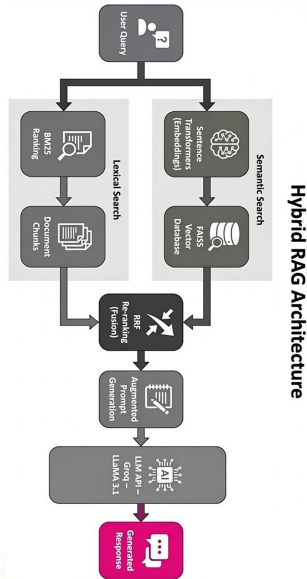




● Réponse Générée (Output)

C'est l'aboutissement final du processus pour l'utilisateur :

- ✉ **Communication :**
La réponse est livrée en langage naturel, fluide et directement exploitable.
- ✓ **Fiabilité :**
Grâce au contexte fourni par le RAG, le message est précis, vérifiable et dépourvu d'hallucinations.





Synthèse

Le système de **RAG Hybride** développé offre une solution robuste pour l'accès aux informations académiques de la FST de Marrakech.

- **🎯 Performance** : Alliances de la précision lexicale (BM25) et de la compréhension sémantique (FAISS).
- **⚡ Rapidité** : Réponses quasi-instantanées grâce à l'infrastructure **Groq**.
- **🔒 Sécurité** : Élimination des hallucinations via un **Prompt Augmenté** rigoureux.

Merci de votre attention !